

Project Title: Spreadsheet Pro

Team: 702

Members: Aidan Domondon, Jamie Li, Abigail Rajadurai, Smita Rosemary

Table of Contents

[Summary of System Functionality](#)

[Core Features](#)

[Additional Features](#)

[#1: Comments:](#)

[#2: Charts](#)

[#3: SQL DDL Interpretation](#)

[System Architecture Overview](#)

[Development Practices](#)

[Reflection](#)

[Installation Instructions and User Guide](#)

Summary of System Functionality

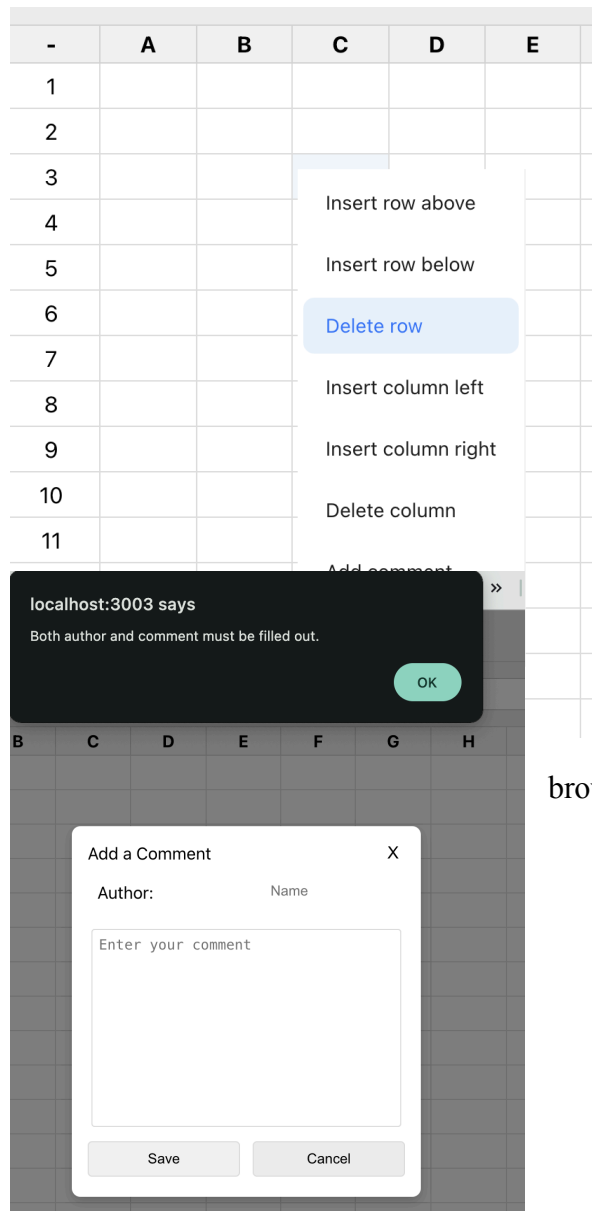
Core Features

Our spreadsheet supports the following **core** features:

- References
- Formulas
 - Basic arithmetic operations
 - SUM & AVERAGE
 - String concatenation
- Left/Right insertion of rows/columns
- Deletion of rows/columns
- Relative addressing
- Error handling

To use any formula, you must enter “=” before the formula. For example, if a user wanted to sum A1 and A2, they would have to type: “=SUM(A1..A2)” in that exact format with two dots representing the range expression. The same is true for string concatenation and strings must be in quotes (e.g. “hello”+”world” to get “helloworld”)

To delete rows, as well as insert them, you must right click on the row you wish to delete or insert above/below. The same is true for inserting and deleting columns. You must right click on the column that you wish to delete or insert left/right of. When right clicking, this menu should appear:



Each menu item corresponds to a different function, as labeled. This React component was made using AntDesign. To close the menu, simply click close. Clicking elsewhere will not close the menu. An option must be selected.

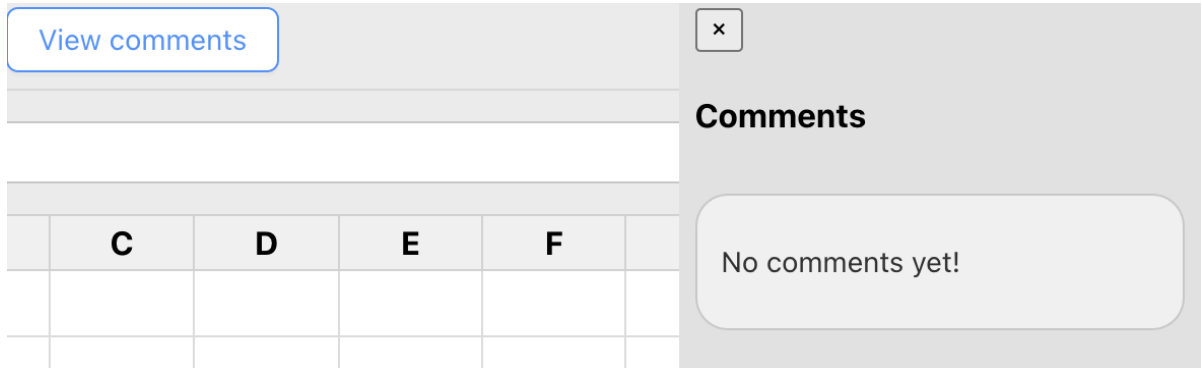
When deleting a row or column, as well as inserting, the cells will shift left or right/up or down depending on the selected menu item.

We also have error handling and prevention, for example, if a user tries to submit a comment with nothing in it, the application will have a browser alert show up with the error message.

Additional Features

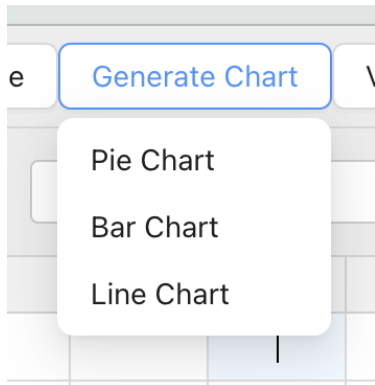
#1: Comments:

- A user can add comments to the spreadsheet. Comments will show up on the right-hand side of the spreadsheet; click the “View comments” button found at the ribbon bar at the top of the screen. Comments have an author and the content of the comment, i.e. the comment itself:

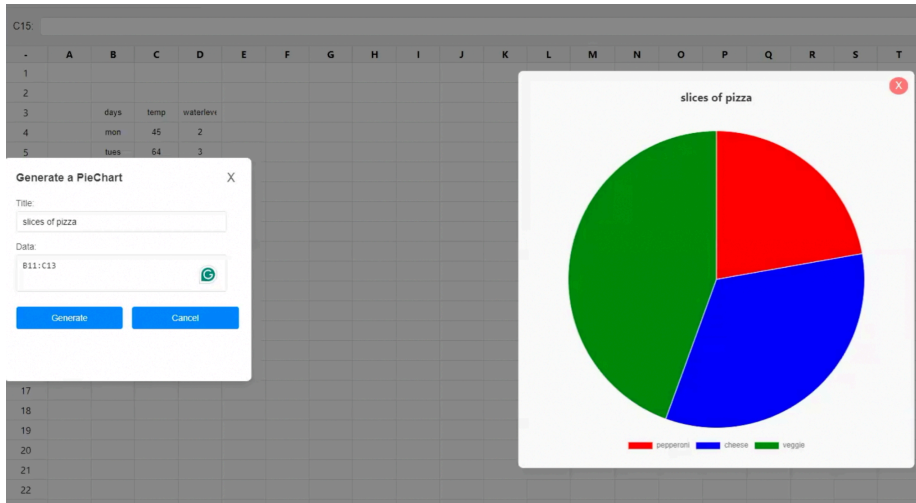
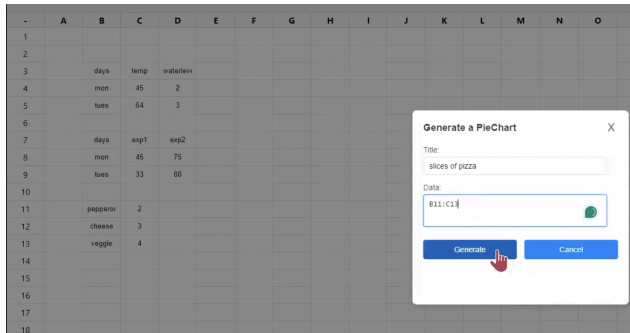


#2: Charts

- Our spreadsheet supports generating three types of charts: bar, line, and pie charts.
- When the user hovers over “Generate Chart”, a drop down menu of three options appears, which the user can then click to select the type of chart they would like to generate

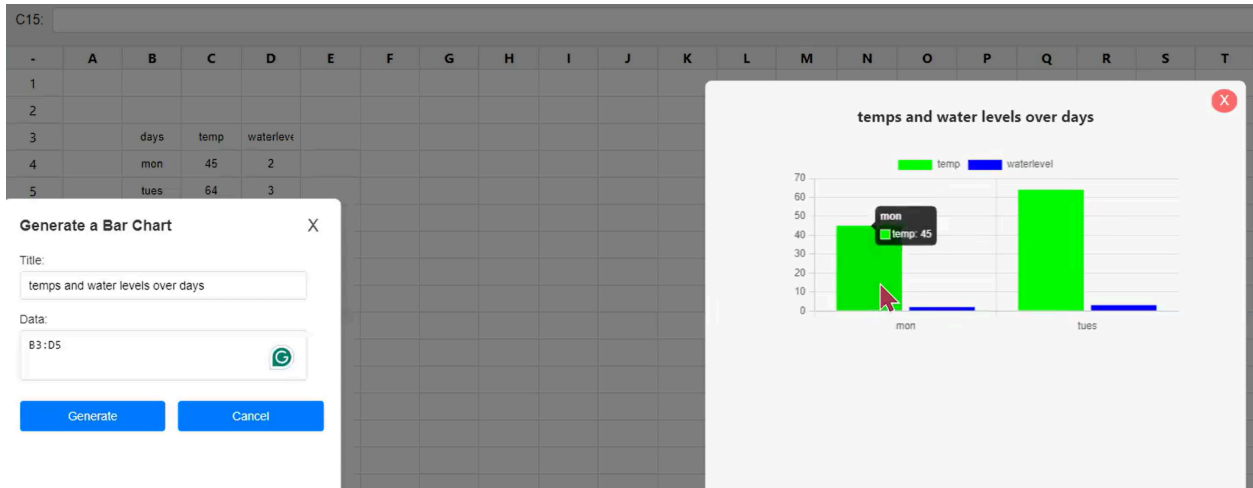


- Below we show how the user can enter in information which the model then uses to create Pie chart data. This chart data is then passed to a component that uses React-chartjs-2 to create and render a pie chart. The same functionality exists for Bar and Line charts. Additionally, the model parses the first column and first row as labels it can use for generating Bar and Line charts! This means when you enter in a table with labels like shown in the below images, the model will use those values as labels, and create a chart accordingly.



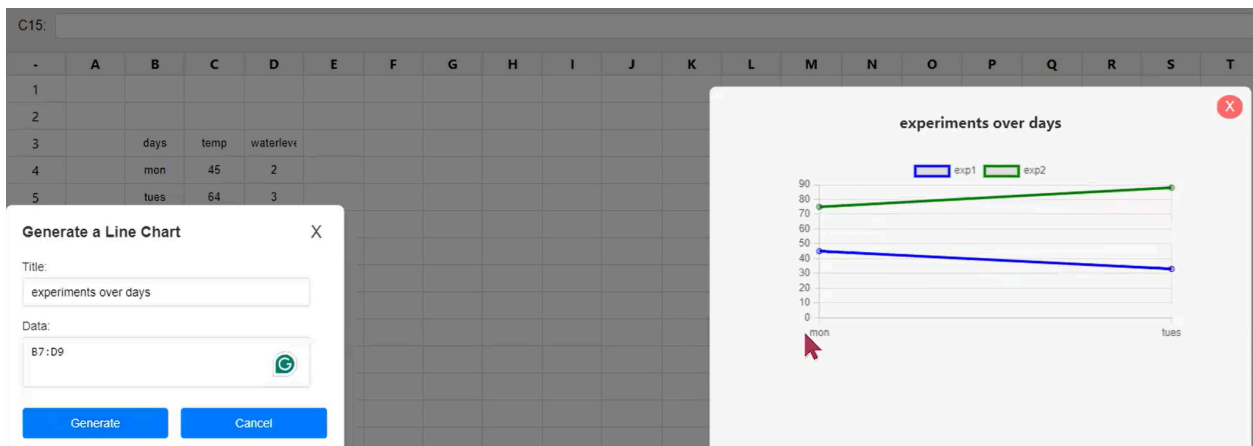
Bar Chart Example:

days	exp1	exp2
mon	45	75
tues	33	88



Line Chart Example:

days	temp	waterlevel
mon	45	2
tues	64	3



#3: SQL DDL Interpretation

Users can populate their spreadsheet with SQLite Data-Definition Language (DDL). The top ribbon includes a hover-field labeled "From SQLite" that, when hovered over reveals a "Interpret SQL" button that brings up the SQL sidebar.

From SQLite
Generate Chart
View

Interpret SQL

-	A	B	C
1			
2			
3			

In this sidebar, users enter a valid SQLite CREATE, potentially followed by **a space** and a valid INSERT INTO query. Upon hitting the “Interpret” button below the text field, the Spreadsheet populates with the appropriate result.

From SQLite
Generate Chart
View comments

A1:

-	A	B	C	D	E
1	"id"	"name"	"age"	"studentN	
2	1	"John"	20	123456	
3	2	"Michael"	45	358	
4	3	"Amy"	52	109	
5					
6					

Interpret SQLite DDL

CREATE TABLE test (id INTEGER PRIMARY KEY, name TEXT, age INTEGER, studentNum INTEGER);
INSERT INTO test VALUES (1, John, 20, 123456), (2, Michael, 45, 358), (3, Amy, 52, 109);

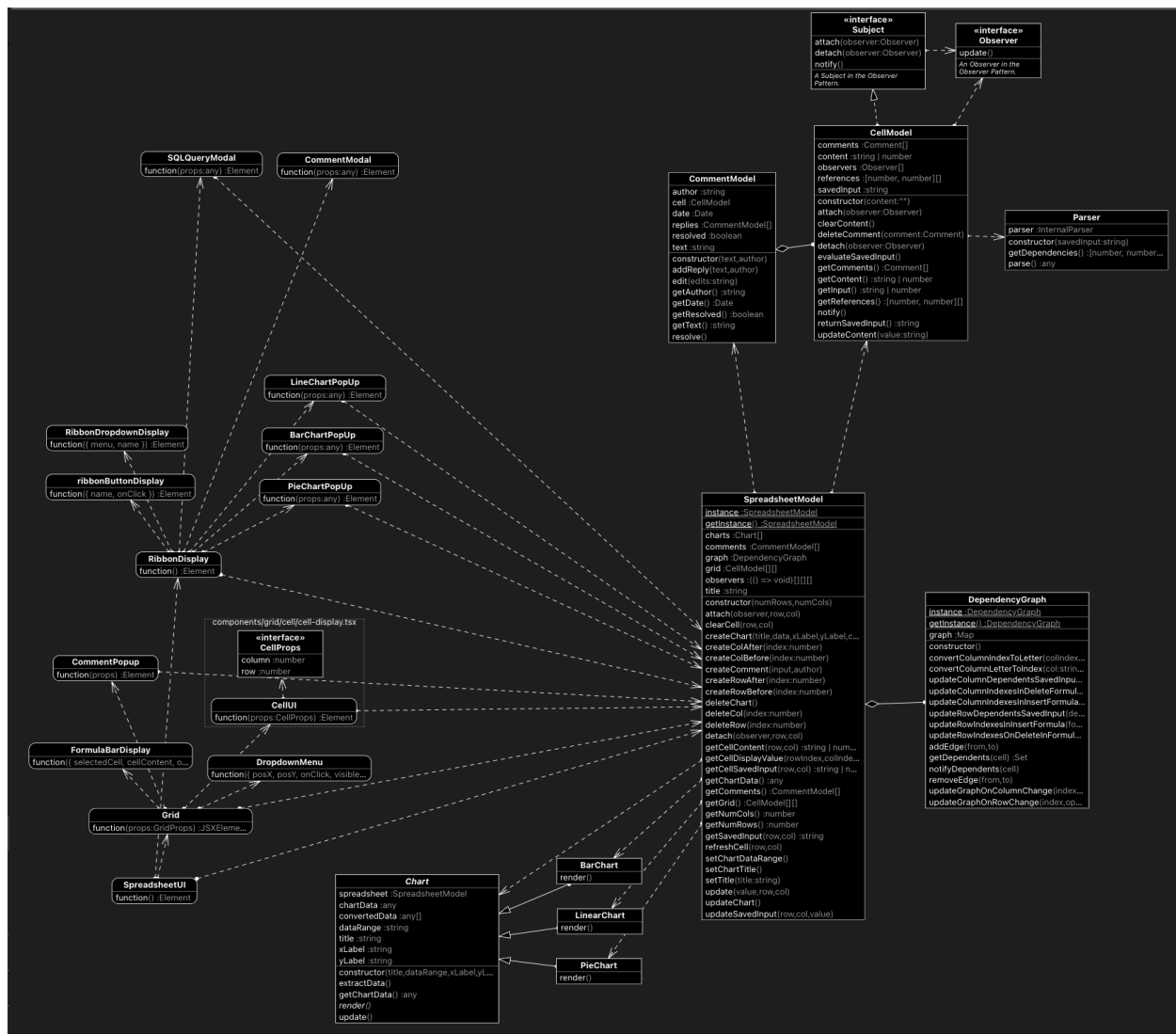
Interpret

System Architecture Overview

The **back-end** state of the spreadsheet is maintained in a Singleton instance of the `SpreadsheetModel` class. Actions on the UI manipulate the back-end spreadsheet data through a singular API with functions to update a certain cell to a certain value, create and delete columns and rows, and others.

The **front-end** UI is organized according to the following hierarchy. The root of the UI is the React Element `SpreadsheetUI` in `Spreadsheet.tsx`, which contains the two halves of the UI– the ribbon and the grid. The ribbon contains buttons to open the SQL interpreter sidebar, to open the chart generator, to open the comment viewer sidebar. The grid is a 2D array of UI cells, each displaying the content of the corresponding cell in the `SpreadsheetModel`.

UML Diagram



Tracking Inter-Cell Dependencies

References are tracked in multiple ways. One of these ways include a dependency graph that is initialized as a singleton that can be called anywhere in the program. Another area that we can get their references is through the parser. First, the parser will see if the saved input (the saved formula) has a REF, SUM, or AVERAGE within it. Then it will go through the string, parsing out all the references within it and storing it in its own list of [row, col]. When any cells are changes/updated on the spreadsheet, the update method is called which stores the input then evaluates it to get the actual content to be displayed. When it is being evaluated, the parser comes into play using the aforementioned logic to maintain a list of references. This list of references is then accessed within the same update method, and an edge is added between each of the references and the current cell (since the current cell technically is a dependent of the references). Then notify dependents is called on an instance of the graph, to notify the dependents of that cell so that its dependents know that the content has changed.

If a cycle is detected (as in it already exists within the graph in reverse order), the cell content is changed to notify the user that they tried to add a cyclic dependency.

E8: =REF(C6)					
-	A	B	C	D	E
1					
2			"#CYCL!"		
3					
4					"#CYCL!"
5					
6			5		
7					
8					5

C	D	E
"#CYCL!"		
		"#CYCL!"

Below shows the functionality when a column is inserted to the left of Column D. As you can see, the formula of the cell that has the SUM updates upon the insert operation. Then, when any of the values used for the SUM is updated, the cell E5 updates with it, indicating that the cell dependency graph was updated according to the insert operation. The same functionality exists for inserting/deleting rows and deleting columns.

D5: =SUM(D2..D4)				
-	A	B	C	D
1				
2				5
3				9
4				7
5				21

E5: =SUM(E2..E4)					
-	A	B	C	D	E
1					
2					5
3					9
4					7
5					21

E5: =SUM(E2..E4)					
-	A	B	C	D	E
1					
2					5
3					89
4					7
5					101

Development Practices

We used GitHub for version control tracking and feature branching. We started out by following some development practices such as using GitHub actions to keep track of tasks on the backlog, but as tasks became more complicated and required more tight collaboration, we often found ourselves resorting to more fluid collaboration rather than strict compartmentalization.

Reflection

Over the course of this project, our group has overcome many obstacles and learned many lessons. Some of those obstacles include the busy schedules of all group members, as well as the introduction of new languages (Typescript and React) and utilizing our resources to assist us in completing our spreadsheet application, Spreadsheet Pro. Some of the lessons we learned include: time management can be difficult (especially among multiple members within a group), unexpected events inevitably occur (but there are valuable resources and support for these occurrences), and making goals (in order to complete tasks before deadlines) is an important part of project development. We also learned that group discussions are very important when completing a project, as everyone needs to be on the same page and communicating effectively. From this, we learned what effective communication can look like: illustrations on a whiteboard, status updates every so often, and many, many group meetings. That said, our group feels as if we could have communicated more and been more efficient if we had utilized asynchronous collaboration. To expand on that point, our group feels as if we could have used GitHub more effectively: i.e., did code reviews, had better branch naming conventions (and naming conventions in general), made use of issues, and more. However, this is not to say that we did not utilize GitHub effectively or too a low level, we were still able to use GitHub to our advantage and in the end, it benefitted us and gave us a new tool to use in future projects: whether for collaboration reasons or version control, or both. Furthermore, this high level organization and communication could help us with our project structure and design as well (it could be more organized).

Installation Instructions and User Guide

Launching Spreadsheet Pro: Instructions

1. Clone the GitHub repository for team 702
2. Open the terminal and type 'npm install'
3. Type 'npm start' in the terminal from the the root of the project
4. Enjoy Spreadsheet Pro!

User Guide

1. Formatting

- a. When using a formula, please use '=' in front of all formulas and references
 - i. References =REF (A1)
 - ii. Range expressions =SUM (A1 . . B2) , =AVERAGE (A1 . . B2)
 - iii. Arithmetic operations =1+2, =2 * 3
 - iv. String concatenation
 1. Strings must be in QUOTES to concatenate
 2. ="hello"+"world"
2. When using the dropdown menu (on right click), please close the menu by clicking the "Close" button.
 - a. Clicking away will not close the menu, an option MUST be selected!!
3. At the top of our spreadsheet application, you will find a ribbon bar with different options, these options include:
 - a. SQL
 - i. Locate the top ribbon, which includes a hover-field labeled "From SQLite" that, when hovered over reveals a "Interpret SQL" button that brings up the SQL sidebar.
 - ii. Enter the SQL.
 - iii. Click "Interpret".
 - iv. Watch your spreadsheet populate!
 - b. Chart generation
 - i. Locate the top ribbon, which includes a hover-field labeled "Generate chart", that when hovered over reveals three buttons– Pie Chart, Bar Chart, and Line Chart, that each open a pop-up wizard to make that type of chart.
 - ii. Fill in a title for the chart in the "Title" input box.
 - iii. Enter a range of cells for the chart to analyze in the "Data" input box.
 1. The syntax uses a colon to delineate a data range (e.g. the area of cells from B2 to D5 is specified by typing in "B2:D5").
 - iv. Click "Generate" to generate the chart on-screen.
 - c. Commenting
 - i. Right click anywhere on the spreadsheet grid, and locate the "Add comment" dropdown menu option.
 - ii. Add a comment, include your name as the author.
 - iii. Click the "View comments" button found at the ribbon bar at the top of the screen.
 - iv. View your comment!